

Python Crash Course E: Data Science und pandas

VU Informatik BW

Thorsten Rupprechter

2 Wiederholung

- Zahlensysteme
- Vorprogrammieren

Kurze Wiederholung: Recursions & File I/O



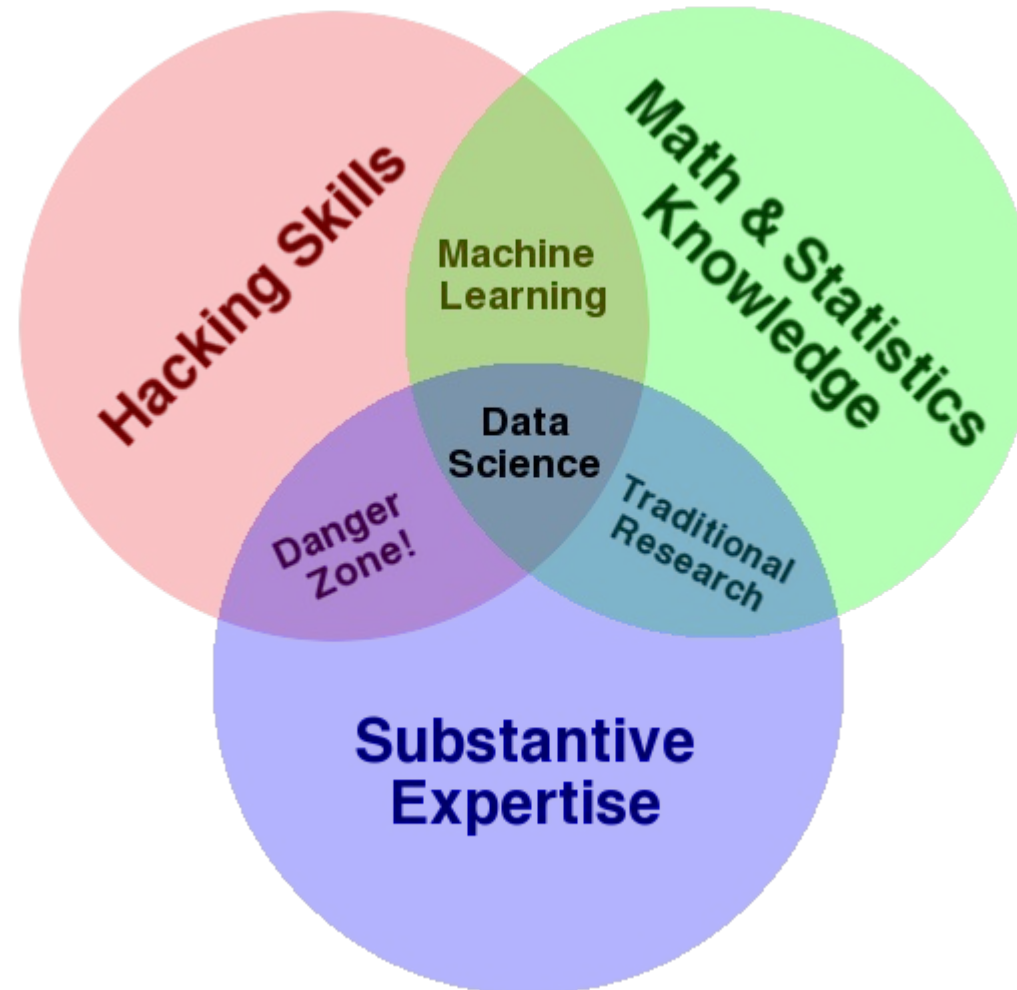
fbr.io/infobw8

Zeit: 2 Minuten



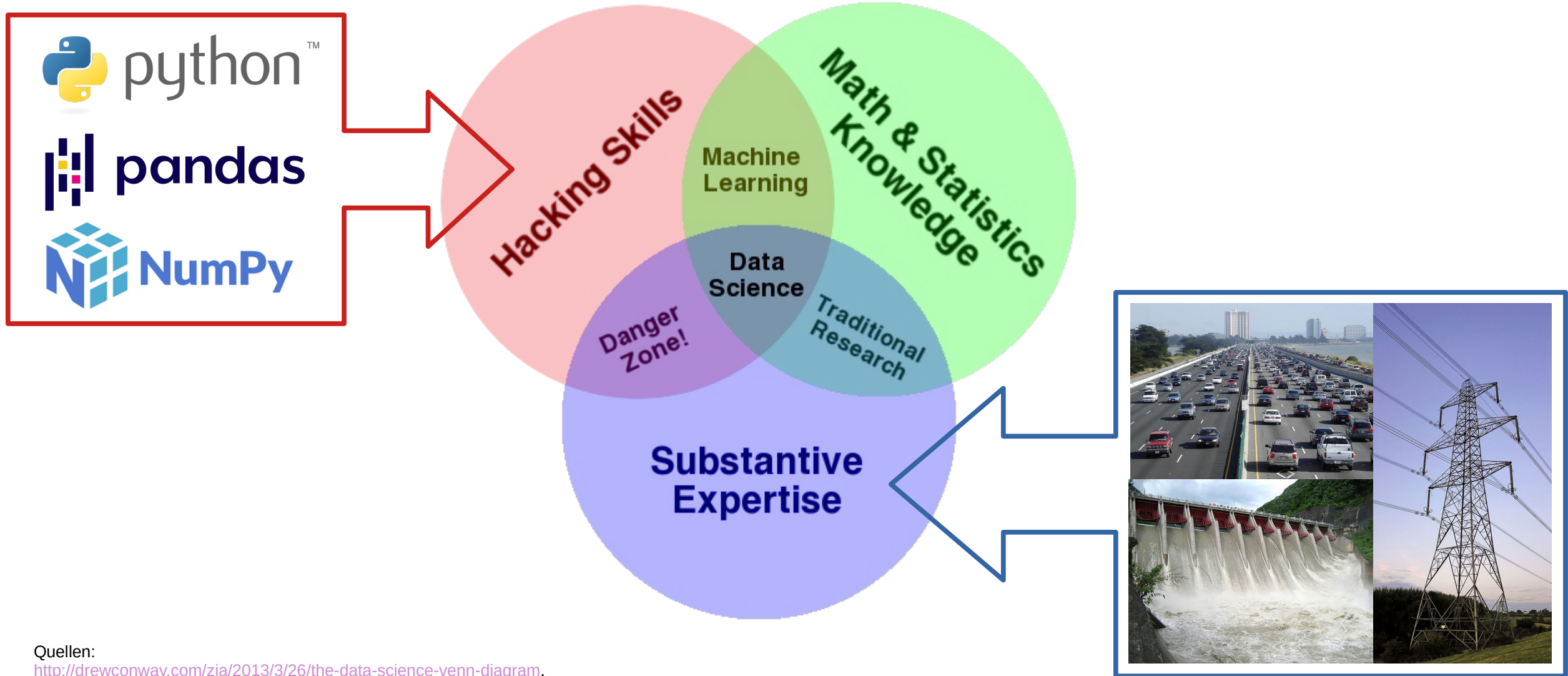
- Intro zu Data Science
- Files: `csv`-Dateien
- Module `import` `t`
- `pandas`-Tutorial
- Organisatorisches & restliche Termine
- Assignment 2

Was ist Data Science?



Quelle: <http://drewconway.com/zia/2013/3/26/the-data-science-venn-diagram>

Data Science in InfoBW



Quellen:

<http://drewconway.com/zia/2013/3/26/the-data-science-venn-diagram>,
Wikimedia Commons

Files: csv-Dateien

- CSV-Dateien = Dateien im Tabellenformat („Excel-Format“)
 - CSV = „Comma Separated Value“
 - „Zellen“ getrennt durch Separator (z. B.: `,`, `;`, `\t`)
 - Standard-Separator abhängig von Betriebssystem und Sprache
 - Können in Tabellenverarbeitungs-Programmen importiert werden
- Lesen/Schreiben von csv-Dateien:
 - Standard File I/O oder externe „Modules“ (z.B. `csv`, `pandas`)

```
Mat.Nr., Vorname, Nachname, Punkte
1234, Martina, Muster, 10
1313, Max, Muster, 12
4242, Test, Student, 16
2424, Test, Studentin, 14
```



	A	B	C	D
1	<u>Mat.Nr.</u>	<u>Vorname</u>	<u>Nachname</u>	<u>Punkte</u>
2	1234	Martina	Muster	10
3	1313	Max	Muster	12
4	4242	Test	Student	16
5	2424	Test	<u>Studentin</u>	14

Module `import` (1)

- `import`-Keyword
 - Importieren von Funktionalität aus anderen Python-Scripts oder Modulen
- Python modules („libraries“): Externe Code-Bibliotheken
 - Funktionalität kann in Programm verwendet werden
 - Module aus [Python Standard Library](#) standardmäßig verfügbar
 - Externe Module (z. B. `pandas`): Installation über `pip`
- Module importieren = „Werkzeug/Tool aus Werkzeugkiste nehmen“

Module `import` (2)

- `import` Syntax
 - `import <ModuleName>`
 - Oder: `import <ModuleName> as <Kürzel>`
 - Beispiele:
 - `import random`
 - `import pandas as pd`
- Aufruf von Funktionen: `<Kürzel>.<FunktionsName>()`

```
# random-module: https://docs.python.org/3/library/random.html  
import random as rd
```

```
# for example, get random number between 1 and (inclusive) 13  
random_number = rd.randint(1, 13)
```

CSV-Dateien laden mit pandas: `pd.read_csv()`

states.csv

In Texteditor

```
state,population,area
California,38332521.0,423967.0
Texas,26448193.0,695662.0
New York,19651127.0,141297.0
Florida,19552860.0,170312.0
New Jersey,9288994.0,
Hawaii,0.0,0.0
Illinois,12882135.0,149995.0
```

In Excel

	A	B	C	
1	state	population	area	
2	California	38332521	423967	
3	Texas	26448193	695662	
4	New York	19651127	141297	
5	Florida	19552860	170312	
6	New Jersey	9288994		
7	Hawaii	0	0	
8	Illinois	12882135	149995	

Code (Jupyter-Notebook)

DataFrame laden

```
[1]: import pandas as pd
df = pd.read_csv('states.csv')
df # or: display(states)
```

DataFrame anzeigen

```
[1]:
```

	state	population	area
0	California	38332521.0	423967.0
1	Texas	26448193.0	695662.0
2	New York	19651127.0	141297.0
3	Florida	19552860.0	170312.0
4	New Jersey	9288994.0	NaN
5	Hawaii	0.0	0.0
6	Illinois	12882135.0	149995.0

pandas: Allgemeines (1)

- Python-Modul für Verarbeitung „tabularer Daten“ (z. B. xls/csv-Tabellen)
 - `import pandas as pd`
- Haupt-Datenstruktur in pandas: `DataFrame`
 - Mehrdimensionales Array/Liste
 - Feste Bezeichnungen für Zeilen und Spalten
 - Zugriff auf Zeilen/Spalten ähnlich Dictionary
 - Einzelne Spalten/Zeilen: Ähnlich zu numpy-Arrays

	population	area
California	38332521	423967
Texas	26448193	695662
New York	19651127	141297
Florida	19552860	170312
Illinois	12882135	149995



pandas: Allgemeines (2)

- pandas und DataFrame
 - Effiziente Datenverwaltung
 - Komfortable und leistungsfähige Funktionalität:
 - Laden und Speichern
 - Zugriff und Filtern
 - Fehlerhafte bzw. fehlende Daten ersetzen oder verwerfen
 - Ergebnisse berechnen
 - Neue Inhalte anlegen
 - Daten Visualisieren (plotten)
 - ...

pandas: DataFrames manuell anlegen

■ Anlegen aus Dictionary

```
import pandas as pd
data = {'population': {'California': 38332521,
                      'New York': 19651127,
                      'Florida': 19552860},
        'area': {'California': 423967,
                  'New York': 141297,
                  'Florida': 170312}}
df = pd.DataFrame(data)
```



	population	area
California	38332521	423967
New York	19651127	141297
Florida	19552860	170312

■ Anlegen aus Liste

```
import pandas as pd
data = [['California', 423967, 38332521],
        ['New York', 141297, 19651127],
        ['Florida', 170312, 19552860]]
df = pd.DataFrame(data, columns=['state', 'area', 'population'])
```



	state	area	population
0	California	423967	38332521
1	New York	141297	19651127
2	Florida	170312	19552860

■ In der Praxis: Laden von Daten aus Tabellenformat (z.B. CSV)

pandas: DataFrame-Zugriff auf Spalten und Zeilen

■ Zugriff auf Spalten (Bsp.: area)

- `df['area']`
- `df.area`
- `df.loc[:, 'area']`

- Zugriff auf Zeilen per Index mit `loc` (Index muss nicht Integer sein)

- `df.loc[3]`

```
df = pd.read_csv('states.csv')
```

	state	population	area
0	California	38332521.0	423967.0
1	Texas	26448193.0	695662.0
2	New York	19651127.0	141297.0
3	Florida	19552860.0	170312.0
4	New Jersey	9288994.0	NaN
5	Hawaii	0.0	0.0
6	Illinois	12882135.0	149995.0

pandas: Filtern von Spalten nach Werten (1)

■ Filtern mit Operatoren

- Vergleichsoperatoren wie gehabt: `>`, `<`, `>=`, `<=`, `==`, `!=`
- Logische Operatoren leicht anders in pandas vs. „normalem“ Python:

“Normal”	pandas
and	<code>&</code>
or	<code> </code>
not	<code>~</code>

- Anwendung von Operator auf DataFrame-Spalte = „Boolean-Maske“
 - Zugriff DataFrame-Spalte mit Boolean-Maske = gefilterte Auswahl

pandas: Filtern von Spalten nach Werten (2)

- Welche Zeilen beinhalten Staaten mit mehr als 25 Millionen Einwohnern?

- `df[df.population > 25000000]`
- `df[df['population'] > 25000000]`
- `df.loc[df.population > 25000000]`

- Warum funktioniert die Syntax? Boolean-Masking!

- `(df.population > 25000000)`

=>

`[True, True, False, False, False, False]`

```
df = pd.read_csv('states.csv')
```

	state	population	area
0	California	38332521.0	423967.0
1	Texas	26448193.0	695662.0
2	New York	19651127.0	141297.0
3	Florida	19552860.0	170312.0
4	New Jersey	9288994.0	NaN
5	Hawaii	0.0	0.0
6	Illinois	12882135.0	149995.0

pandas: Filtern von Spalten nach Werten (3)

- Welche Zeilen beinhalten Staaten mit mehr als 15 Millionen Einwohnern und weniger als 200 Tausend km² Fläche?

```
• df[(df.population > 15000000) &
      (df.area < 200000)]
• df.loc[(df.population > 15000000) &
          (df.area < 200000)]
```

- Was sind die Namen dieser Staaten (in 'state')?

```
• df[(df.population > 15000000) &
      (df.area < 200000)]['state']
• df.loc[(df.population > 15000000) &
          (df.area < 200000), 'state']
```

```
df = pd.read_csv('states.csv')
```

	state	population	area
0	California	38332521.0	423967.0
1	Texas	26448193.0	695662.0
2	New York	19651127.0	141297.0
3	Florida	19552860.0	170312.0
4	New Jersey	9288994.0	NaN
5	Hawaii	0.0	0.0
6	Illinois	12882135.0	149995.0

pandas: String-Operationen und Filtern von String-Spalten

- String-Operationen in pandas: per `str`-Attribut von Spalten aufrufbar
 - Ähnlich zu bekannten String-Funktionen:
 - `str.lower()`, `str.upper()`, `str.contains()`,
`str.startswith()`, `str.endswith()`, `str.split()`, ...
 - Alle Funktionen: siehe [pandas-Dokumentation](#)

- Beispiel: Wähle alle Staaten mit `New` im Namen aus.

```
• df[df['state'].str.contains('New')]  
• df[df.state.str.contains('New')]
```

DataFrame df			
	state	population	area
0	California	38332521.0	423967.0
1	Texas	26448193.0	695662.0
2	New York	19651127.0	141297.0
3	Florida	19552860.0	170312.0
4	New Jersey	9288994.0	NaN
5	Hawaii	0.0	0.0
6	Illinois	12882135.0	149995.0

pandas: DataFrames und Spalten aggregieren

- Operation über gesamte Spalte oder DataFrame berechnen
- pandas-Methoden ähnlich zu Listen-Funktionen
 - Listen:
 - **Funktionen:** `sum()`, `max()`, `min()`, `len()`, ...
 - Beispiel mit Liste `elements`: `len(elements)`
 - pandas:
 - **Methoden:** `sum()`, `max()`, `min()`, `count()`,
`median()`, `mean()`, `value_counts()`, ...
 - Beispiel DataFrame-Spalte: `df['area'].sum()`

pandas: Arithmetische Spaltenoperationen

- Arithmetische Operatoren auf Spalten anwendbar
 - Siehe Standard-Python: `*`, `/`, `+`, `-`, ...
 - Ergebnis wieder „Spalte“ (`Series`-Objekt)
 - Im Hintergrund: `numpy`-array
- Bsp.: Berechne area in „squared miles“ statt km^2 (Faktor 0,386).

```
df['area'] * 0.386
df.area * 0.386
```

- Direkt kombinierbar mit Aggregation (Bsp. Summe)
 - `(df['area'] * 0.386).sum()`
 - Ergebnis: 610355.938

DataFrame df

	state	population	area
0	California	38332521.0	423967.0
1	Texas	26448193.0	695662.0
2	New York	19651127.0	141297.0
3	Florida	19552860.0	170312.0
4	New Jersey	9288994.0	NaN
5	Hawaii	0.0	0.0
6	Illinois	12882135.0	149995.0

```
0    163651.262
1    268525.532
2     54540.642
3     65740.432
4           NaN
5           0.000
6     57898.070
Name: area, dtype: float64
```

Output in Notebook

pandas: Fehlende oder fehlerhafte Daten (1)

- Fehlende Daten in geladenen Datensätzen: Zellen mit `NaN`
 - `NaN` = Spezielle pandas-Repräsentation von Python's `None`
- Automatisches Ausfüllen von `NaN` mit Wert (`value`):
 - In gesamtem `DataFrame` (`df`)
 - `df = df.fillna(value)`
 - Nur in bestimmter Spalte (Bsp.: mit Spaltenname `column`)
 - `df['column'] = df['column'].fillna(value)`
 - `df = df.fillna({'column': value})`

pandas: Fehlende oder fehlerhafte Daten (2)

- Fülle alle fehlenden Werte in der `area` Spalte mit einem Default-Wert von 25000 km² auf.

```
• df['area'] = df['area'].fillna(25000)
• df.loc[:, 'area'] = df['area'].fillna(25000)
• df = df.fillna({'area': 25000})
```

```
df = pd.read_csv('states.csv')
```

	state	population	area
0	California	38332521.0	423967.0
1	Texas	26448193.0	695662.0
2	New York	19651127.0	141297.0
3	Florida	19552860.0	170312.0
4	New Jersey	9288994.0	NaN
5	Hawaii	0.0	0.0
6	Illinois	12882135.0	149995.0

pandas: Fehlende oder fehlerhafte Daten (2)

- Fülle alle fehlenden Werte in der `area` Spalte mit einem Default-Wert von 25000 km² auf.

```
• df['area'] = df['area'].fillna(25000)
• df.loc[:, 'area'] = df['area'].fillna(25000)
• df = df.fillna({'area': 25000})
```

- Exkludiere alle Zeilen mit fehlerhaften Werten (= 0) in `population` oder `area`.
= „Wähle alle Zeilen mit `area` und `population` > 0“

```
• df = df[(df.area > 0) &
          (df.population > 0)]
• df = df.loc[(df.area > 0) &
              (df.population > 0)]
```

```
df = pd.read_csv('states.csv')
```

	state	population	area
0	California	38332521.0	423967.0
1	Texas	26448193.0	695662.0
2	New York	19651127.0	141297.0
3	Florida	19552860.0	170312.0
4	New Jersey	9288994.0	25000.0
5	Hawaii	0.0	0.0
6	Illinois	12882135.0	149995.0

pandas: Neue Spalten anlegen (1)

- Neue Spalte = Eintragen eines neuen Key-Value-Pairs bei Dictionary
 - Selber Wert (value) für alle Zeilen
 - `df['new_column'] = value` # Kürzer.
 - `df.loc[:, 'new_column'] = value` # Sicherer!
 - Aus Operation mit bestehender Spalte
 - `df['pop_in_mil'] = df['population'] / 1000000`
 - `df.loc[:, 'pop_in_mil'] = df['population'] / 1000000`
 - Aus Operation mit mehreren bestehenden Spalten
 - `df['pop_density'] = df['population'] / df['area']`
 - `df.loc[:, 'pop_density'] = df['population'] / df['area']`

pandas: Neue Spalten anlegen (2)

```
df.loc[:, 'pop_in_mil'] = df['population'] / 1000000
# Can also use Python math operations like round(), abs(), ...
df.loc[:, 'pop_in_mil'] = round(df['pop_in_mil'])
df.loc[:, 'pop_density'] = round(df['population'] / df['area'], 2)
```

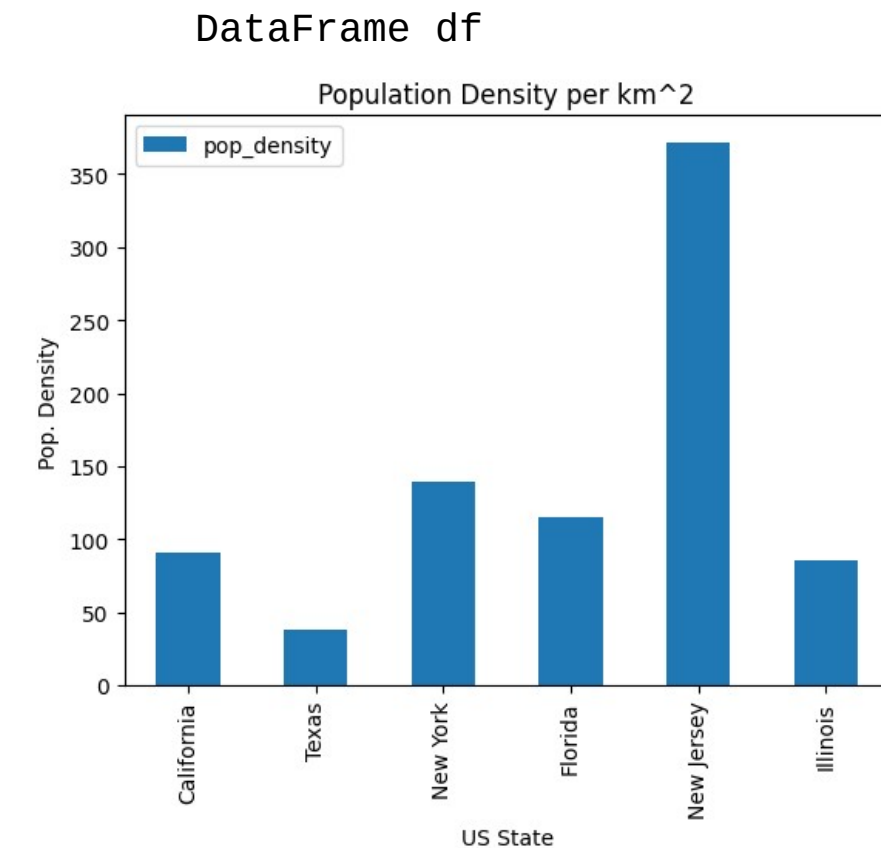
	state	population	area	pop_in_mil	pop_density
0	California	38332521.0	423967.0	38.0	90.41
1	Texas	26448193.0	695662.0	26.0	38.02
2	New York	19651127.0	141297.0	20.0	139.08
3	Florida	19552860.0	170312.0	20.0	114.81
4	New Jersey	9288994.0	25000.0	9.0	371.56
6	Illinois	12882135.0	149995.0	13.0	85.88

pandas: Plotting

	state	population	area	pop_in_mil	pop_density
0	California	38332521.0	423967.0	38.0	90.41
1	Texas	26448193.0	695662.0	26.0	38.02
2	New York	19651127.0	141297.0	20.0	139.08
3	Florida	19552860.0	170312.0	20.0	114.81
4	New Jersey	9288994.0	25000.0	9.0	371.56
6	Illinois	12882135.0	149995.0	13.0	85.88

- Simple Plotting-Schnittstelle über `df.plot()`
 - Siehe [Pandas-Referenz](#)
 - Beispiel: Population-Density Bar-Chart

```
df.plot(x='state', y='pop_density',
        title='Population Density per km^2',
        kind='bar', xlabel='US State',
        ylabel='Pop. Density')
```



pandas: DataFrames als CSV-Datei speichern

- Speichern über DataFrame-Methode `to_csv(path)`
 - `path` = Ausgabepfad, Dateiname
 - Optionaler Parameter `index` (Boolean) ob Index in Ausgabe
- `df.to_csv('states_processed.csv', index=False):`

	state	population	area	pop_in_mil	pop_density
0	California	38332521.0	423967.0	38.0	90.41
1	Texas	26448193.0	695662.0	26.0	38.02
2	New York	19651127.0	141297.0	20.0	139.08
3	Florida	19552860.0	170312.0	20.0	114.81
4	New Jersey	9288994.0	25000.0	9.0	371.56
6	Illinois	12882135.0	149995.0	13.0	85.88

DataFrame **df**



```
state,population,area,pop_in_mil,pop_density
California,38332521.0,423967.0,38.0,90.41
Texas,26448193.0,695662.0,26.0,38.02
New York,19651127.0,141297.0,20.0,139.08
Florida,19552860.0,170312.0,20.0,114.81
New Jersey,9288994.0,25000.0,9.0,371.56
Illinois,12882135.0,149995.0,13.0,85.88
```

states_processed.csv (in Texteditor)

pandas: DataFrames als CSV-Datei speichern

- Speichern über DataFrame-Methode `to_csv(path)`
 - `path` = Ausgabepfad, Dateiname
 - Optionaler Parameter `index` (Boolean) ob Index in Ausgabe
- `df.to_csv('states_processed.csv', index=False):`

	state	population	area	pop_in_mil	pop_density
0	California	38332521.0	423967.0	38.0	90.41
1	Texas	26448193.0	695662.0	26.0	38.02
2	New York	19651127.0	141297.0	20.0	139.08
3	Florida	19552860.0	170312.0	20.0	114.81
4	New Jersey	9288994.0	25000.0	9.0	371.56
6	Illinois	12882135.0	149995.0	13.0	85.88

DataFrame **df**



	A	B	C	D	E
1	state	population	area	pop_in_mil	pop_density
2	California	38332521	423967	38	90.41
3	Texas	26448193	695662	26	38.02
4	New York	19651127	141297	20	139.08
5	Florida	19552860	170312	20	114.81
6	New Jersey	9288994	25000	9	371.56
7	Illinois	12882135	149995	13	85.88

states_processed.csv (in Excel)

- Siehe Jupyter-Notebooks
 - `lecture.ipynb`: Beispiele aus diesen Slides
 - `datascience.ipynb`: Erweitertes Beispiel für eine Data Science Pipeline mit gesamtem Datensatz über US-Staaten

(Beide Notebooks enthalten hilfreiche Code-Snippets für Assignment 3)

Fragen?

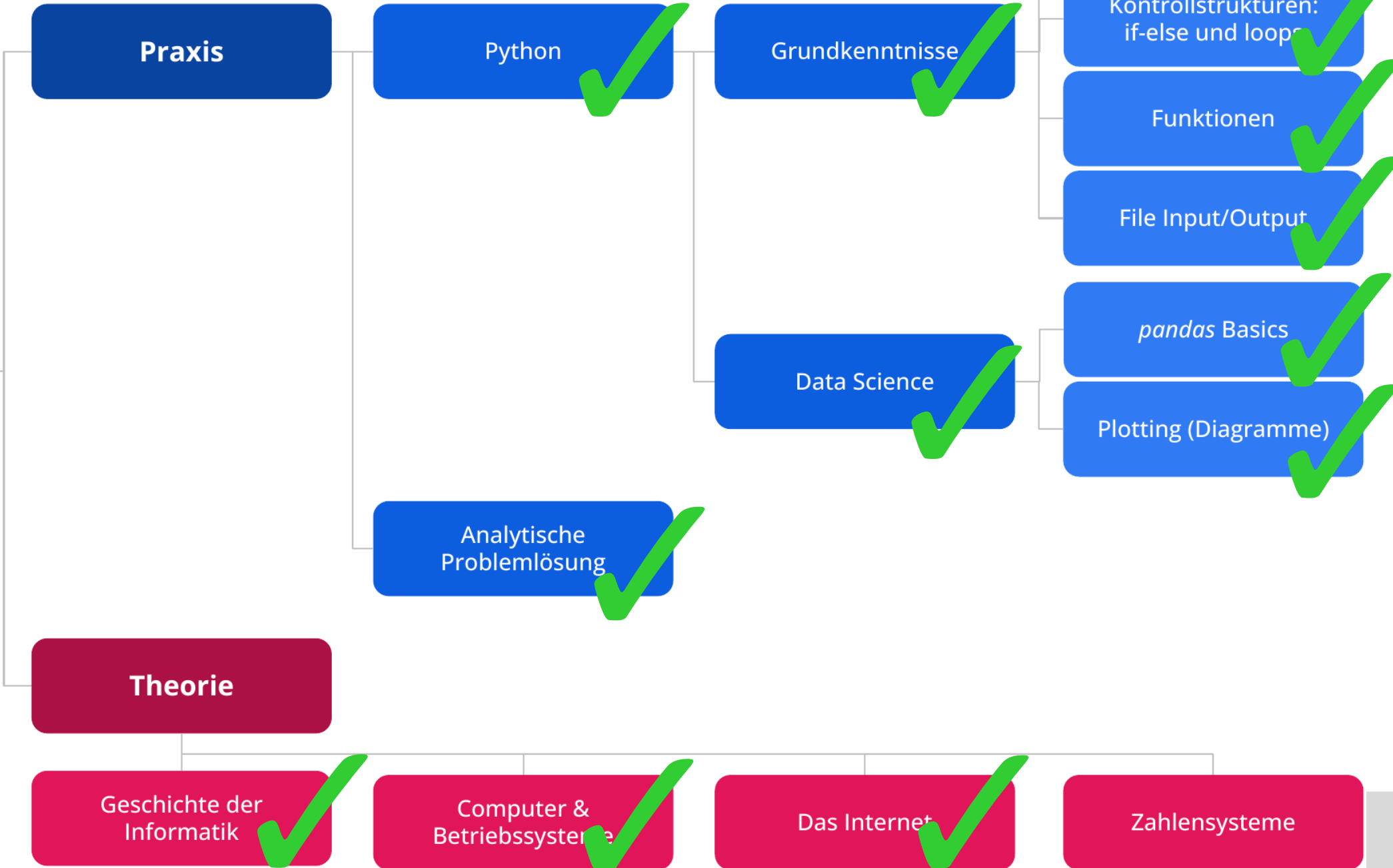
Organisatorisches:

Verbleibende VU Outline (mit Vorbehalt, ohne Tutorien)

Typ	Datum	Details
UE	(Heute) 28.11.	Ausgabe Ass3
VO	5. 12.	Zahlensysteme (ff.) und Binärrechnen
VO	12. 12.	Klausurvorbereitung
UE	15. 12.	Deadline Ass3
AG	07. 01. – 08. 01.	Abgabegespräche Ass2
K	09. 01.	Klausur (Online, Open-Book)
NK	16. 01.	Nachklausur (bei negativer Hauptklausur)

- 04.12. und 11.12:
Tutorien (pandas, Fragestunde Ass2)

Hellgrau = Übung
 Dunkelgrau = Vorlesung
Schwarz = Anwesenheitspflicht (AG, VP, K)
Fett = Deadlines Assignments



Assignment 2:

Data Science for Civil Engineers

(25 Punkte)

Assignment 2: Inhalte

- Beispiel für Data Science in “Ihrem” Anwendungsfeld:
Daten über Vorarlberger Skilifte und Seilbahnen
 - Einführung in Data Science mit `pandas`
- Separate Teile einer Data-Science-Pipeline:
 - Daten laden: 1 Punkte
 - Preprocessing: 4 Punkte
 - Exploration und Visualisierung: 7,5 Punkte
 - Analyse und Berechnungen: 12,5 Punkte**=> 25 Punkte**

Assignment 2: Abgabe

- Exakte Beschreibung in `assignment2_angabe.html`
- Deadline: 15.12.2024 (23:59), Abgabedatei `assignment2.ipynb`
- Hinweis (siehe Angabe):
 - Anforderungen und Output sind EXAKT wiederzugeben
 - Kommentar-Header muss in beiden Dateien enthalten sein
 - Coding-Standard muss eingehalten werden

Assignment 2: Automatisierte Tests

- Automatisiertes Testing für **Teil der Funktionalität**
 - **Alle Tests erfolgreich $\neq$ Alle Punkte!**
 - Für finale Abgabe: Mehr Testcases
- Test-Runs jeden Dienstag- und Freitag-Abend
 - Grundlage für Tests: Abgaben vor 18:00
- Testreport per automatischer Mail bis Mittwoch-/Samstagmorgen (~9:00)
 - Fragen zu Testreport: TeachCenter/Tutorium!

Python Crash Course E: Data Science und pandas

VU Informatik BW

Thorsten Ruprechter